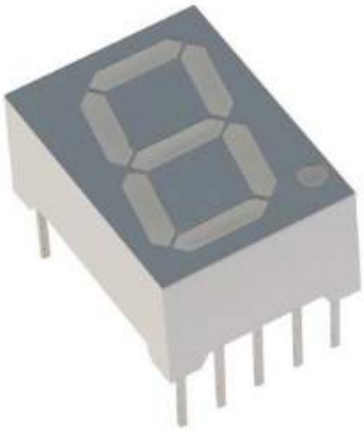
부록 2
PyTorch의 이해와 활용

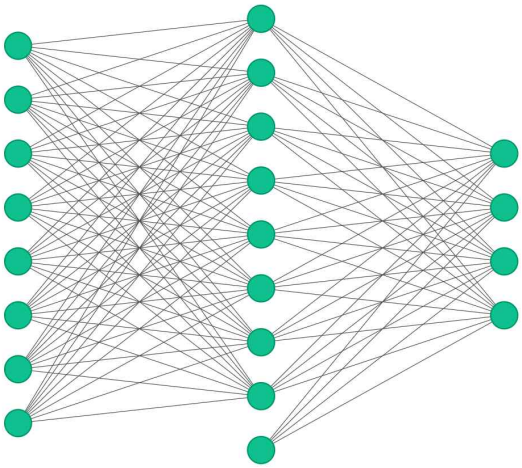
본문에서 독자 여러분은 Tensorflow 사용법을 살펴보았습니다. 여기서는 PyTorch 사용법을 소개합니다. PyTorch는 Tensorflow와 함께 가장 많이 사용되는 인공지능 프레임워크입니다. 여기서는 딥러닝 7 공식을 PyTorch로 구현해 보며 PyTorch의 내부적인 동작을 이해해 봅니다. PyTorch의 내부 동작을 잘 이해하여 PyTorch의 활용도를 높일 수 있도록 합니다.



01 PyTorch 활용하기

여기서는 PyTorch를 활용하여 7 segment에 대한 인공 신경망을 학습시켜 봅니다. 이 과정에서 PyTorch 신경망에 적용할 수 있는 입력 데이터와 출력 데이터의 형식을 이해하고 활용할 수 있도록 합니다.



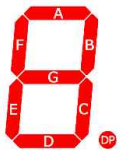


01 7 세그먼트 인공 신경망

여기서는 7 세그먼트에 숫자 값에 따라 표시되는 LED의 ON, OFF 값을 입력으로 받아 2 진수로 출력하는 인공 신경망을 구성하고 학습시켜 봅니다. 다음은 7 세그먼트 디스플레이 2 진수 연결 진리표입니다.

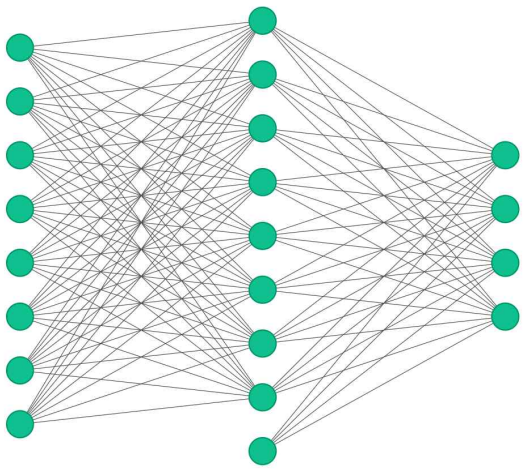
7 세그먼트 2 진수 연결 진리표

	A	B	C	D	E	F	G				
	In	In	In	In	In	In	In	Out	Out	Out	Out
0	1	1	1	1	1	1	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	1
2	1	1	0	1	1	0	1	0	0	1	0
3	1	1	1	1	0	0	1	0	0	1	1
4	0	1	1	0	0	1	1	0	1	0	0
5	1	0	1	1	0	1	1	0	1	0	1
6	0	0	1	1	1	1	1	0	1	1	0
7	1	1	1	0	0	0	0	0	1	1	1
8	1	1	1	1	1	1	1	1	0	0	0
9	1	1	1	0	0	1	1	1	0	0	1



 = 1011011 ➡ 0101

그림에서 7 세그먼트에 5로 표시되기 위해 7개의 LED가 1011011(1-ON, 0-OFF)의 비트열에 맞춰 켜지거나 꺼져야 합니다. 해당 비트열에 대응하는 이진수는 0101입니다. 여기서는 다음 그림과 같이 7개의 입력, 8개의 은닉층, 4개의 출력층으로 구성된 인공 신경망을 학습시켜 봅니다.



입력층, 은닉층의 맨 하단의 노드는 편향 노드입니다.

torch.FloatTensor로 데이터 초기화하기

먼저 입력값과 목표값을 torch.FloatTensor로 초기화합니다.

1. 다음과 같이 예제를 작성합니다.
_7seg_data_torch.py

```
01 import torch
02
03 torch.set_printoptions(precision=4, linewidth=120, sci_mode=False)
04
05 X=torch.FloatTensor([
06     [ 1, 1, 1, 1, 1, 1, 0 ],  # 0
07     [ 0, 1, 1, 0, 0, 0, 0 ],  # 1
08     [ 1, 1, 0, 1, 1, 0, 1 ],  # 2
09     [ 1, 1, 1, 1, 0, 0, 1 ],  # 3
10     [ 0, 1, 1, 0, 0, 1, 1 ],  # 4
11     [ 1, 0, 1, 1, 0, 1, 1 ],  # 5
12     [ 0, 0, 1, 1, 1, 1, 1 ],  # 6
13     [ 1, 1, 1, 0, 0, 0, 0 ],  # 7
14     [ 1, 1, 1, 1, 1, 1, 1 ],  # 8
15     [ 1, 1, 1, 0, 0, 1, 1 ]   # 9
16 ])
17 YT=torch.FloatTensor([
18     [ 0, 0, 0, 0 ],
19     [ 0, 0, 0, 1 ],
20     [ 0, 0, 1, 0 ],
21     [ 0, 0, 1, 1 ],
22     [ 0, 1, 0, 0 ],
23     [ 0, 1, 0, 1 ],
24     [ 0, 1, 1, 0 ],
25     [ 0, 1, 1, 1 ],
26     [ 1, 0, 0, 0 ],
27     [ 1, 0, 0, 1 ]
28 ])
```

03 : torch.set_printoptions 함수를 호출하여 print 함수로 torch Tensor를 출력할 때, 실수 출력 형식을 조절합니다. precision 인자는 소수점 이하 자리수를 설정하며, 여기서는 소수점 이하 4 자리까지 출력합니다. linewidth 인자는 한 줄에 표시할 수 있는 문자수로 120으로 설정합니다. sci_mode 인자는 실수 표기 방식을 설정하며, False로 설정할 경우 고정 소수점 표기법(Fixed point notation)을 사용하며, True로 설정할 경우 과학적 표기법(Scientific notation)을 사용합니다. 예를 들어, 0.000514는 고정 소수점 표기법이고, 5.14e-04는 과학적 표기법입니다. 여기서는 고정 소수점 표기법을 사용합니다.

05~16 : X 변수를 선언하고, 진리표의 입력 값으로 초기화된 2차 실수 텐서를 할당합니다.
17~28 : YT 변수를 선언하고, 진리표의 목표 값으로 초기화된 2차 실수 텐서를 할당합니다.

2. 계속해서 다음과 같이 예제를 작성합니다.
_631.py

```
01 from _7seg_data_torch import X, YT
02
03 print(X.shape)
04 print(YT.shape)
```

01 : _7seg_data_torch 모듈로부터 X, YT를 불러옵니다.
03 : X의 모양을 출력합니다.
04 : YT의 모양을 출력합니다.

3. 다음과 같이 예제를 실행합니다.

```
$ python _631.py
```

다음은 실행 결과 화면입니다.

```
torch.Size([10, 7])
torch.Size([10, 4])
```

X는 10*7 크기의 2차 배열입니다.
YT는 10*4 크기의 2차 배열입니다.

딥러닝 모델 학습시키기

다음은 딥러닝 모델을 생성한 후, 학습을 시켜 봅니다.

1. 다음과 같이 예제를 작성합니다.
_631_2.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04 from _7seg_data_torch import X, YT
05
06 model=nn.Sequential(
07       nn.Linear(7,8),
08       nn.ReLU(),
09       nn.Linear(8,4),
10       nn.Sigmoid()
11 )
12 print(model)
13
14 loss_fn = nn.MSELoss()
15 optimizer = optim.Adam(model.parameters(), lr=0.001)
16
17 for epoch in range(10000):
18
19       Y = model(X) # 1공식, 순전파
20       E = loss_fn(Y, YT) # 2공식, 오차계산
21       optimizer.zero_grad()
22       E.backward() # 6공식, 오차역전파
23       optimizer.step() # 7공식, 학습
24
25       if epoch%100 == 99:
26            print(epoch, E.item())
27
28 Y = model(X)
29 print(Y.data)
```

06~11 : 입력층의 노드 수는 7개, 은닉층의 노드 수는 8개, 출력층의 노드 수는 4개인 신경망 모델을 생성합니다. 은닉 층의 활성화 함수는 ReLU, 출력 층의 활성화 함수는 Sigmoid로 설정합니다.
13 : 오차 함수를 nn.MSELoss로 설정합니다. nn.MSELoss는 평균 제곱 오차 함수입니다.
14 : 학습 함수를 optim.Adam으로 설정합니다. optim.Adam은 가장 많이 사용하는 학습 함수입니다.
16 : 학습을 10000 회 수행합니다.
28 : X에 대해 예측을 수행해 봅니다.

29 : 예측 값 Y를 출력합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _631_2.py
```

다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=7, out_features=8, bias=True)
  (1): ReLU()
  (2): Linear(in_features=8, out_features=4, bias=True)
  (3): Sigmoid()
)
9999 1.4533863577526063e-05
tensor([[    0.0049,     0.0011,     0.0000,     0.0032],
        [    0.0000,     0.0068,     0.0065,     0.9963],
        [    0.0044,     0.0000,     0.9992,     0.0017],
        [    0.0000,     0.0042,     0.9959,     1.0000],
        [    0.0044,     0.9937,     0.0004,     0.0036],
        [    0.0024,     0.9948,     0.0057,     0.9960],
        [    0.0011,     0.9994,     0.9959,     0.0000],
        [    0.0000,     0.9928,     0.9937,     1.0000],
        [    0.9929,     0.0008,     0.0030,     0.0000],
        [    0.9972,     0.0067,     0.0001,     0.9968]])
```

YT값과 비교해 봅니다. YT값에 가깝게 학습된 것을 볼 수 있습니다.

학습이 잘 안 되는 경우도 있습니다. 다음은 4에 대한 학습이 제대로 안된 경우입니다.

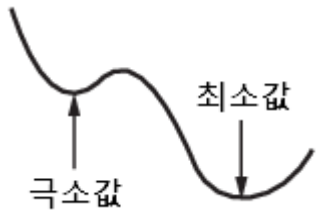
```
9999 0.0500122606754303
tensor([[    0.0058,     0.0035,     0.0009,     0.0034],
        [    0.0000,     0.5001,     0.0051,     0.9952],
        [    0.0039,     0.0000,     0.9949,     0.0052],
        [    0.0012,     0.0010,     0.9978,     0.9999],
        [    0.0058,     0.5002,     0.0000,     0.0054],
        [    0.0043,     0.9989,     0.0000,     0.9983],
        [    0.0015,     0.9937,     0.0000,     0.0000],
        [    0.0015,     0.4999,     0.9948,     1.0000],
        [    0.9916,     0.0008,     0.0005,     0.0003],
        [    0.9956,     0.5000,     0.0046,     0.9939]])
```

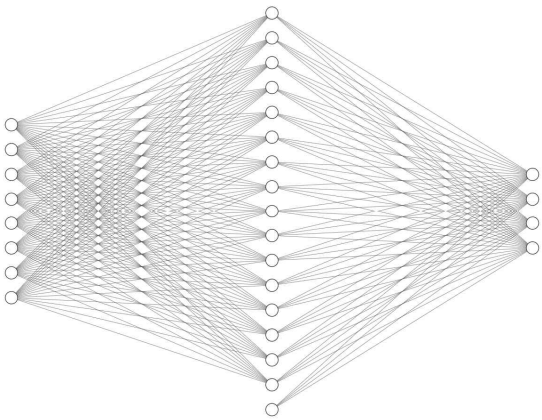
다음은 7에 대한 학습이 제대로 안된 경우입니다.

```
9999 0.025008762255311012
tensor([[    0.0046,     0.0000,     0.0000,     0.0032],
        [    0.0000,     0.0030,     0.0054,     0.9959],
        [    0.0043,     0.0000,     0.9999,     0.0016],
        [    0.0010,     0.0000,     0.9964,     0.9999],
        [    0.0036,     0.9957,     0.0000,     0.0033],
        [    0.0035,     0.9984,     0.0037,     0.9971],
        [    0.0004,     1.0000,     0.9952,     0.0000],
        [    0.0010,     0.0000,     0.9947,     1.0000],
        [    0.9936,     0.0032,     0.0046,     0.0003],
        [    0.9962,     0.0026,     0.0000,     0.9979]])
```

국소해의 문제 해결해 보기

앞의 예제에서 은닉층 노드의 개수가 8일 경우 학습이 제대로 되지 않는 경우가 있는데 이런 현상은 국소해의 문제로 발생합니다. 예를 들어, 다음 그림에서 신경망의 학습 과정에서 최소값 지점을 찾지 못하고 극소값 지점에 수렴하는 경우입니다. 국소해의 문제가 발생할 경우엔 재학습을 수행해 보거나 은닉층의 노드수를 변경해 봅니다. 여기서는 은닉층 노드의 개수를 16으로 늘려봅니다.





1. 다음과 같이 예제를 수정합니다.

_631_3.py

```
01~05 # 이전 예제와 같습니다.
06 model=nn.Sequential(
07       nn.Linear(7,16),
08       nn.ReLU(),
09       nn.Linear(16,4),
10       nn.Sigmoid()
11 )
12 print(model)
13~끝 # 이전 예제와 같습니다.
```

07,09 : 은닉층이 노드 개수를 16으로 늘립니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _631_3.py
```

다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=7, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=4, bias=True)
  (3): Sigmoid()
)
```

```
9999 4.2585488699842244e-06
tensor([[    0.0030,     0.0010,     0.0004,     0.0018],
        [    0.0000,     0.0030,     0.0027,     0.9972],
        [    0.0027,     0.0000,     0.9992,     0.0010],
        [    0.0000,     0.0025,     0.9975,     0.9997],
        [    0.0027,     0.9973,     0.0009,     0.0026],
        [    0.0023,     0.9978,     0.0025,     0.9984],
        [    0.0007,     1.0000,     0.9970,     0.0000],
        [    0.0000,     0.9966,     0.9978,     1.0000],
        [    0.9958,     0.0004,     0.0025,     0.0002],
        [    0.9971,     0.0025,     0.0000,     0.9979]])
```

출력층에 linear 함수 적용해 보기

여기서는 출력층 함수에 linear 함수를 적용해 봅니다. linear 함수는 출력단의 값을 그대로 내보내는 함수입니다. linear 함수를 적용하여 학습을 수행할 경우 선형 회귀라고 합니다.

1. 다음과 같이 예제를 수정합니다.

_631_4.py

```
01~05 # 이전 예제와 같습니다.
06 model=nn.Sequential(
07       nn.Linear(7,16),
08       nn.ReLU(),
09       nn.Linear(16,4),
10       #nn.Sigmoid()
11 )
12 print(model)
13~끝 # 이전 예제와 같습니다.
```

10 : 출력층의 활성화 함수를 주석 처리합니다. 이 경우 09줄의 출력값이 그대로 나오게 됩니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _631_4.py
```

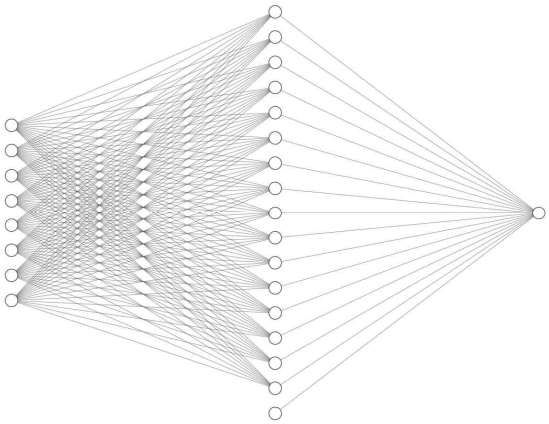
다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=7, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=4, bias=True)
)
9999 8.206879728751837e-13
tensor([[    0.0000,    -0.0000,     0.0000,     0.0000],
        [    0.0000,    -0.0000,     0.0000,     1.0000],
        [    0.0000,    -0.0000,     1.0000,     0.0000],
        [    0.0000,    -0.0000,     1.0000,     1.0000],
        [    0.0000,     1.0000,     0.0000,     0.0000],
        [    0.0000,     1.0000,     0.0000,     1.0000],
        [    0.0000,     1.0000,     1.0000,     0.0000],
        [    0.0000,     1.0000,     1.0000,     1.0000],
        [    1.0000,    -0.0000,    -0.0000,     0.0000],
        [    1.0000,    -0.0000,     0.0000,     1.0000]])
```

loss(오차)가 이전 예제보다 더 줄어드는 것을 확인합니다.

목표값 변경해 보기

여기서는 목표값의 형식을 2진수에서 10진수로 변경해 봅니다. 신경망 출력층 노드 개수가 다음과 같이 1개가 됩니다.



1. 다음과 같이 _7seg_data_torch.py 라이브러리를 수정합니다.

_7seg_data_torch.py

```
01 import torch
02
03 torch.set_printoptions(precision=4, linewidth=120, sci_mode=False)
04
05 X=torch.FloatTensor([
06       [ 1, 1, 1, 1, 1, 1, 0 ],  # 0
07       [ 0, 1, 1, 0, 0, 0, 0 ],  # 1
08       [ 1, 1, 0, 1, 1, 0, 1 ],  # 2
09       [ 1, 1, 1, 1, 0, 0, 1 ],  # 3
10       [ 0, 1, 1, 0, 0, 1, 1 ],  # 4
11       [ 1, 0, 1, 1, 0, 1, 1 ],  # 5
12       [ 0, 0, 1, 1, 1, 1, 1 ],  # 6
13       [ 1, 1, 1, 0, 0, 0, 0 ],  # 7
14       [ 1, 1, 1, 1, 1, 1, 1 ],  # 8
15       [ 1, 1, 1, 0, 0, 1, 1 ]   # 9
16 ])
17 YT=torch.FloatTensor([
18       [ 0, 0, 0, 0 ],
19       [ 0, 0, 0, 1 ],
20       [ 0, 0, 1, 0 ],
21       [ 0, 0, 1, 1 ],
22       [ 0, 1, 0, 0 ],
23       [ 0, 1, 0, 1 ],
```

```
24      [ 0, 1, 1, 0 ],
25      [ 0, 1, 1, 1 ],
26      [ 1, 0, 0, 0 ],
27      [ 1, 0, 0, 1 ]
28 ])
29 YT_1=torch.FloatTensor([
30      [0],
31      [1],
32      [2],
33      [3],
34      [4],
35      [5],
36      [6],
37      [7],
38      [8],
39      [9]
40 ])
```

29~40 : YT_1 변수를 선언하고, 10진수 형식의 목표값으로 초기화된 1차 numpy 배열을 할당합니다.
을 합니다.

2. 다음과 같이 예제를 수정합니다.
_631_5.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04 from _7seg_data_torch import X, YT_1
05
06 YT = YT_1
07
08 model=nn.Sequential(
09      nn.Linear(7,16),
10      nn.ReLU(),
11      nn.Linear(16,1),
12      #nn.Sigmoid()
13 )
14 print(model)
15~끝 # 이전 예제와 같습니다.
```

04 : _7seg_data 모듈로부터 X, YT_1을 불러옵니다.
06 : YT 변수를 선언하고, YT_1로 초기화합니다.
11 : 출력층 노드의 개수를 1개로 줄입니다.

3. 다음과 같이 예제를 실행합니다.

```
$ python _631_5.py
```

다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=7, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=1, bias=True)
)
9999 6.00147698464476e-13
tensor([[    -0.0000],
        [     1.0000],
        [     2.0000],
        [     3.0000],
        [     4.0000],
        [     5.0000],
        [     6.0000],
        [     7.0000],
        [     8.0000],
        [     9.0000]])
```

결과가 잘 나오는 것을 확인합니다. 결과가 잘 안 나올 경우 재학습을 수행합니다.

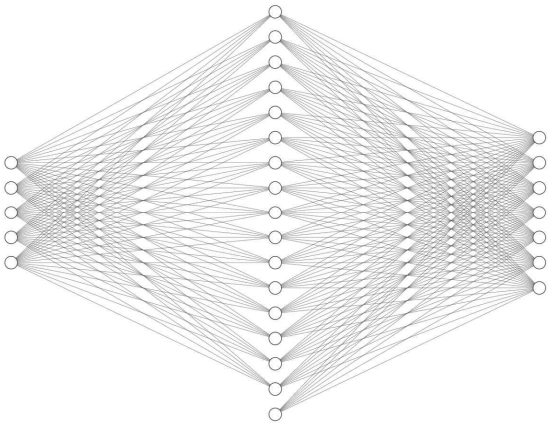
입력층과 목표층 바꿔보기

다음은 이전 예제의 입력층과 목표층을 바꿔 인공 신경망을 학습 시켜봅니다. 다음과 같이 2진수가 입력되면 해당되는 7 세그먼트의 켜지고 꺼져야 할 LED의 비트열을 출력합니다.

2 진수 7 세그먼트 연결 진리표

In	In	In	In	Out	Out	Out	Out	Out	Out	Out
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	0	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	1	1

0101 ➡ 1011011 = 



예를 들어, "숫자 5에 맞게 7 세그먼트 LED를 켜줘!" 하고 싶을 때, 사용할 수 있는 인공 신경망입니다.

1. 다음과 같이 예제를 수정합니다.

_631_6.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04 from _7seg_data_torch import X, YT
05
06 X, YT = YT, X
07
08 model=nn.Sequential(
09       nn.Linear(4,16),
10       nn.ReLU(),
11       nn.Linear(16,7),
12 #       nn.Sigmoid()
13 )
14 print(model)
15~끝 # 이전 예제와 같습니다.
```

04 : _7seg_data 모듈로부터 X, YT을 불러옵니다.

06 : YT, X를 X, YT로 변경합니다. 즉, 입력과 출력을 바꿔줍니다.

09 : 입력층 노드의 개수를 4로 바꿉니다.

11 : 출력층 노드의 개수를 7로 바꿉니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _631_6.py
```

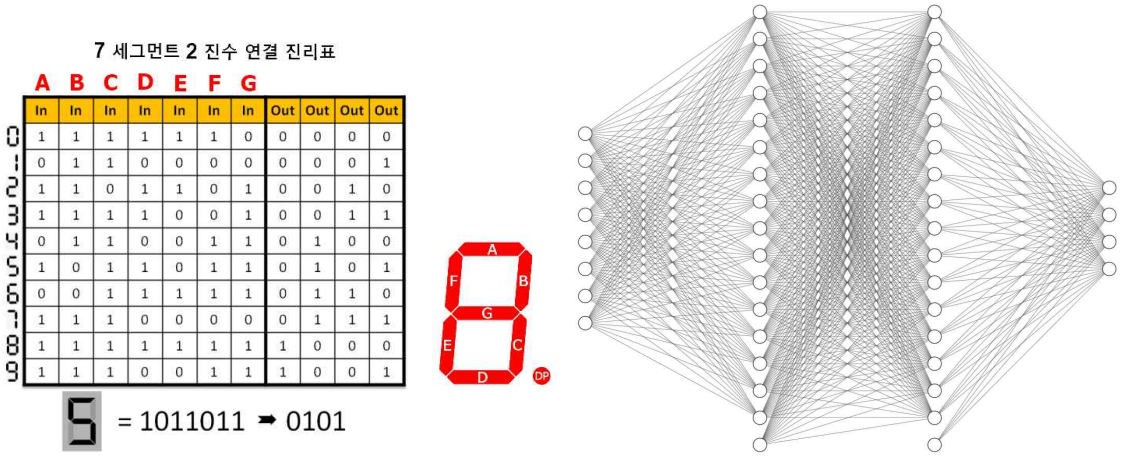
다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=4, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=7, bias=True)
)
```

```
9999 1.30421871219477713e-11
tensor([[     1.0000,      1.0000,      1.0000,      1.0000,      1.0000,      1.0000,     -0.0000],
        [    -0.0000,      1.0000,      1.0000,     -0.0000,      0.0000,     -0.0000,     -0.0000],
        [     1.0000,      1.0000,     -0.0000,      1.0000,      1.0000,     -0.0000,      1.0000],
        [     1.0000,      1.0000,      1.0000,      1.0000,      0.0000,     -0.0000,      1.0000],
        [    -0.0000,      1.0000,      1.0000,     -0.0000,     -0.0000,      1.0000,      1.0000],
        [     1.0000,      0.0000,      1.0000,      1.0000,      0.0000,      1.0000,      1.0000],
        [    -0.0000,      0.0000,      1.0000,      1.0000,      1.0000,      1.0000,      1.0000],
        [     1.0000,      1.0000,      1.0000,      0.0000,      0.0000,     -0.0000,     -0.0000],
        [     1.0000,      1.0000,      1.0000,      1.0000,      1.0000,      1.0000,      1.0000],
        [     1.0000,      1.0000,      1.0000,      0.0000,      0.0000,      1.0000,      1.0000]])
```

02 은닉층 늘려보기

여기서는 은닉층을 늘려 봅니다. 일반적으로 은닉층의 개수가 2개 이상일 때 심층 인공 신경망이라고 합니다. 데이터는 다음과 같이 원래 데이터를 사용합니다.



7 세그먼트 2 진수 연결 진리표

	A	B	C	D	E	F	G	Out	Out	Out	Out
0	1	1	1	1	1	1	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	1
2	1	1	0	1	1	0	1	0	0	1	0
3	1	1	1	1	0	0	1	0	0	1	1
4	0	1	1	0	0	1	1	0	1	0	0
5	1	0	1	1	0	1	1	0	1	0	1
6	0	0	1	1	1	1	1	0	1	1	0
7	1	1	1	0	0	0	0	0	1	1	1
8	1	1	1	1	1	1	1	1	0	0	0
9	1	1	1	0	0	1	1	1	0	0	1

= 1011011 ➡ 0101

1. 다음과 같이 예제를 작성합니다.
_632.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04 from _7seg_data_torch import X, YT
05
06 model=nn.Sequential(
07         nn.Linear(7,16),
08         nn.ReLU(),
09         nn.Linear(16,16),
10         nn.ReLU(),
11         nn.Linear(16,4),
12         nn.Sigmoid()
13 )
14 print(model)
15~끝 # 이전 예제와 같습니다.
```

09 : 은닉층을 하나 더 늘립니다. 노드의 개수는 16으로 합니다.
10 : nn.ReLU 함수를 추가합니다.
12 : 출력층의 활성화 함수가 Sigmoid인 것을 확인합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _632.py
```

다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=7, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=16, bias=True)
  (3): ReLU()
  (4): Linear(in_features=16, out_features=4, bias=True)
  (5): Sigmoid()
)
```

```
9999 4.979730192644638e-07
tensor([[    0.0011,     0.0000,     0.0007,     0.0008],
        [    0.0000,     0.0011,     0.0007,     0.9991],
        [    0.0007,     0.0000,     0.9998,     0.0003],
        [    0.0000,     0.0008,     0.9991,     1.0000],
        [    0.0010,     0.9992,     0.0005,     0.0009],
        [    0.0008,     0.9993,     0.0011,     0.9992],
        [    0.0000,     1.0000,     0.9991,     0.0000],
        [    0.0000,     0.9987,     0.9992,     1.0000],
        [    0.9987,     0.0000,     0.0001,     0.0000],
        [    0.9990,     0.0008,     0.0000,     0.9991]])
```

학습 시키고 모델 내보내기

다음은 인공 신경망을 준비하여 학습을 수행한 후, 수행 결과를 저장합니다.

1. 다음과 같이 예제를 수정합니다. 이전 예제의 마지막에 다음과 같이 추가합니다.
_632_2.py

```
01~31 # 이전 예제와 같습니다.
32
33 torch.save(model, 'model.pth')
```

33 : torch.save 함수를 이용하여 학습된 인공 신경망을 'model.pth' 파일로 저장해 줍니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _632_2.py
```

3. 학습이 끝난 후, 다음과 같이 model.h5 파일이 생성된 것을 확인합니다.

```
linaro@linaro-alip:~/pyLabs$ ls -l model.pth
-rw-r--r-- 1 linaro linaro 4423 Feb  9 05:09 model.pth
```

출력 결과도 확인합니다. 바로 다음 예제의 결과와 비교하여 같은 결과를 얻어야 합니다.

```
9999 4.841845111513976e-07
tensor([[    0.0012,     0.0007,     0.0011,     0.0008],
        [    0.0006,     0.0010,     0.0007,     0.9993],
        [    0.0008,     0.0000,     0.9994,     0.0002],
        [    0.0000,     0.0009,     0.9996,     0.9997],
        [    0.0010,     0.9990,     0.0007,     0.0006],
        [    0.0004,     0.9990,     0.0002,     0.9992],
        [    0.0000,     0.9998,     0.9991,     0.0000],
        [    0.0000,     0.9988,     0.9992,     1.0000],
        [    0.9986,     0.0000,     0.0000,     0.0004],
        [    0.9992,     0.0010,     0.0000,     0.9996]])
```

모델 불러와 예측하기 1

다음은 학습된 인공 신경망를 불러와 예측을 수행해 봅니다.

1. 다음과 같이 예제를 작성합니다.

_632_3.py

```
1 import torch
2 from _7seg_data_torch import X
3
4 model = torch.load('model.pth')
5 print(model)
6
7 Y = model(X)
8 print(Y.data)
```

4 : torch.load 함수를 호출하여 학습된 모델을 불러옵니다.

5 : 모델을 출력합니다.

7 : 예측을 수행합니다.

8 : 예측 결과를 출력합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _632_3.py
```

다음은 실행 결과 화면입니다. 바로 전 예제 결과와 같습니다.

```
Sequential(
  (0): Linear(in_features=7, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=16, bias=True)
  (3): ReLU()
  (4): Linear(in_features=16, out_features=4, bias=True)
  (5): Sigmoid()
)
tensor([[    0.0012,     0.0007,     0.0011,     0.0008],
        [    0.0006,     0.0010,     0.0007,     0.9993],
        [    0.0008,     0.0000,     0.9994,     0.0002],
        [    0.0000,     0.0009,     0.9996,     0.9997],
        [    0.0010,     0.9990,     0.0007,     0.0006],
        [    0.0004,     0.9990,     0.0002,     0.9992],
        [    0.0000,     0.9998,     0.9991,     0.0000],
        [    0.0000,     0.9988,     0.9992,     1.0000],
        [    0.9986,     0.0000,     0.0000,     0.0004],
        [    0.9992,     0.0010,     0.0000,     0.9996]])
```

모델 불러와 예측하기 2

여기서는 1개 데이터에 대한 예측을 해 봅니다.

다음과 같이 예제를 수정합니다.
_632_4.py

```
1 import torch
2 from _7seg_data_torch import X
3
4 model = torch.load('model.pth')
5 print(model)
6
7 Y = model(X[:1])
8 print(Y.data)
```

7 : 입력을 X[:1]로 변경합니다. 이렇게 하면 X의 0번 항목으로만 구성된 텐서가 됩니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _632_4.py
```

다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=7, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=16, bias=True)
  (3): ReLU()
  (4): Linear(in_features=16, out_features=4, bias=True)
  (5): Sigmoid()
)
tensor([[0.0012, 0.0007, 0.0011, 0.0008]])
```

모델 모양 살펴보기

여기서는 학습된 모델 모양을 살펴봅니다.

다음과 같이 예제를 수정합니다.
_412_5.py

```
01 import torch
02
03 model = torch.load('model_7seg.pth')
04
05 W1, b1 = model[0].weight.data, model[0].bias.data
06 W2, b2 = model[2].weight.data, model[2].bias.data
07 W3, b3 = model[4].weight.data, model[4].bias.data
08
09 print(W1.shape, b1.shape)
10 print(W2.shape, b2.shape)
11 print(W3.shape, b3.shape)
```

05 : model[0].weight.data, model[0].bias.data를 읽어와 각각 W1, b1에 할당합니다.
06 : model[2].weight.data, model[2].bias.data를 읽어와 각각 W2, b2에 할당합니다.
07 : model[4].weight.data, model[4].bias.data를 읽어와 각각 W3, b3에 할당합니다.
09 : print 함수를 호출하여 W1의 모양과 b1의 모양을 출력합니다.
10 : print 함수를 호출하여 W2의 모양과 b2의 모양을 출력합니다.
11 : print 함수를 호출하여 W3의 모양과 b3의 모양을 출력합니다.

2.  예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
torch.Size([16, 7]) torch.Size([16])
torch.Size([16, 16]) torch.Size([16])
torch.Size([4, 16]) torch.Size([4])
```

모델 아두이노 스케치로 변환하기

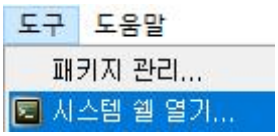
다음은 학습된 모델을 아두이노 스케치에서 사용할 수 있도록 변환합니다.

1. 다음과 같이 예제를 수정합니다.
_412_6.py

```
01 import torch
02
03 model = torch.load('model_7seg.pth')
04
05 W1, b1 = model[0].weight.data, model[0].bias.data
06 W2, b2 = model[2].weight.data, model[2].bias.data
07 W3, b3 = model[4].weight.data, model[4].bias.data
08
09 W1 = torch.transpose(W1, 0, 1)
10 W2 = torch.transpose(W2, 0, 1)
11 W3 = torch.transpose(W3, 0, 1)
12
13 names = ["_W1", "_B1", "_W2", "_B2", "_W3", "_B3"]
14 arrays = [W1, b1, W2, b2, W3, b3]
15
16 # for ESP32 Arduino
17 for name, array in zip(names, arrays):
18       print('double %s[] = {' %name)
19       print('    ', ', '.join([str(x) for x in array.flatten().tolist()]))
20       print('};\n')
```

9~11 : torch.transpose 함수를 호출하여 텐서를 전치해 줍니다. 전치는 행과 열을 바꾸는 것을 말하며 아두이노 스케치에서 사용하기 위한 변환 작업 중 하나입니다.
13 : names 변수를 선언한 후, 가중치와 편향의 이름으로 초기화합니다.
14 : arrays 변수를 선언한 후, 학습된 가중치와 편향으로 초기화합니다.
17 : zip 함수를 이용하여 names와 arrays의 각 항목을 튜플 형태로 묶은 후, 튜플의 각 항목을 name과 array로 받아서 18~20줄을 수행합니다.
18 : 실수 형의 배열 이름과 배열 기호를 출력합니다.
19 : array의 항목들을 문자열 배열로 만든 후, join 함수를 호출하여 쉼표(,)를 구분자로 하나의 문자열로 묶은 후, 출력합니다. flatten 함수를 이용하여 1차 텐서로 변경한 후, tolist 함수를 이용하여 리스트로 변경합니다.
20 : 중괄호와 반점을 출력합니다.

2. [도구]--[시스템 쉘 열기...] 메뉴를 선택합니다.



3. 다음과 같이 명령을 수행합니다.

```
C:₩Users₩edu₩Desktop₩pyLabs>python _412_6.py
```

4. 다음은 출력결과입니다.



가중치와 편향 값이 C/C++ 배열 형태로 변경되었습니다. C/C++ 배열 형태로 변환된 가중치와 편향값은 바로 다음 단원의 아두이노 스케치에서 사용됩니다. 출력 결과를 복사합니다. 우스 왼쪽 버튼을 누른 채로 출력 결과 전체를 긁은 후, ctrl+c 버튼을 누르면 운영체제 내부 버퍼로 복사됩니다.

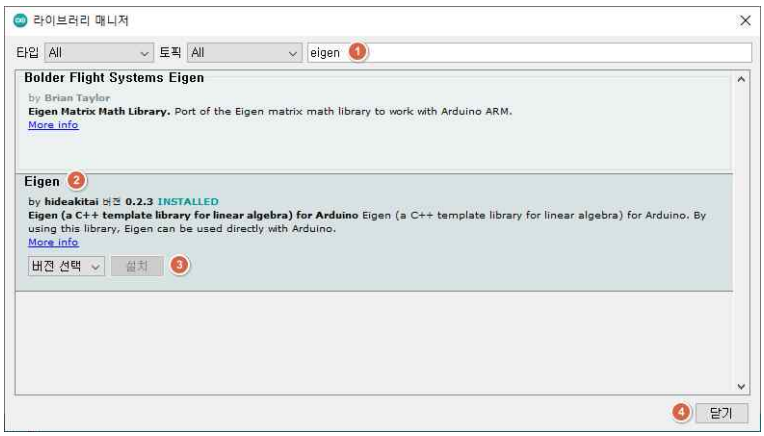
Eigen 행렬 라이브러리 설치하기

아두이노 환경에서는 Eigen 라이브러리를 이용하여 행렬 연산을 수행할 수 있습니다. 파이썬에서 행렬 연산을 지원하는 NumPy 라이브러리도 내부적으로는 Eigen 라이브러리를 사용합니다. 다음과 같은 순서로 Eigen 라이브러리를 설치합니다.

1. 다음과 같이 아두이노 소프트웨어의 [스케치]--[라이브러리 포함하기]--[라이브러리 관리...] 메뉴를 선택합니다.

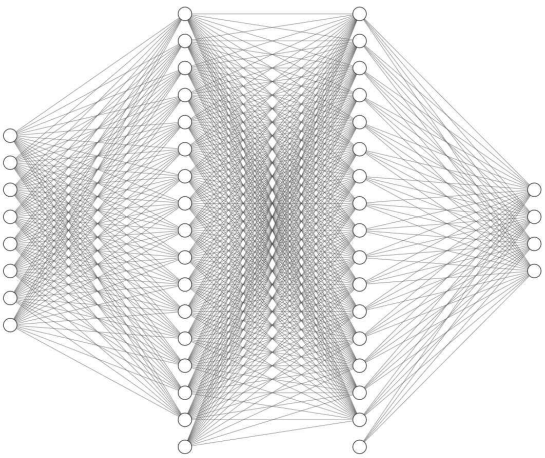


2. 다음과 같이 [라이브러리 매니저] 창에서 [eigen]을 검색한 후, 설치해 줍니다.



아두이노 DNN으로 예측해 보기

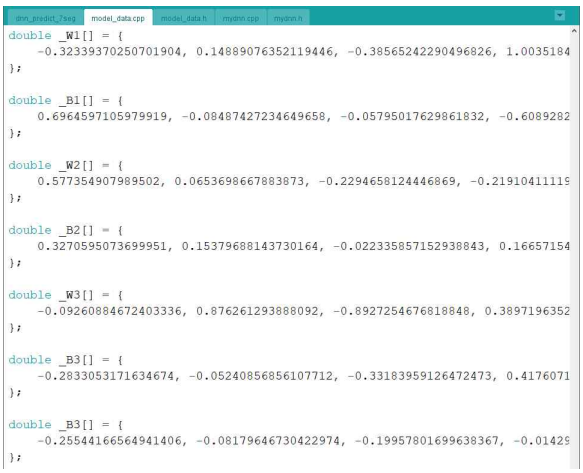
여기서는 아두이노 스케치로 변환된 모델을 이용하여 아두이노에서 예측을 수행해 봅니다. 다음은 아두이노 스케치에 미리 구성된 인공 신경망으로 앞에서 학습된 모델의 구조와 같은 형태입니다.



1. 제공되는 소스에서 다음 예제를 엽니다.



2. 예제가 열리면 model_data.cpp 파일을 열어 앞에서 복사한 가중치와 편향 값을 다음과 같이 덮어씁니다. _W1에서 _B3까지 덮어씁니다.



3. 

업로드를 수행합니다.

4. [시리얼 모니터] 버튼을 클릭하고, 통신 속도를 115200으로 맞춰줍니다.



5. 출력 결과를 확인합니다.

```
4713
0:[[0.0012, 0.0007, 0.0011, 0.0008]]
1:[[0.0006, 0.0010, 0.0007, 0.9993]]
2:[[0.0008, 0.0000, 0.9994, 0.0002]]
3:[[0.0000, 0.0009, 0.9996, 0.9997]]
4:[[0.0010, 0.9990, 0.0007, 0.0006]]
5:[[0.0004, 0.9990, 0.0002, 0.9992]]
6:[[0.0000, 0.9998, 0.9991, 0.0000]]
7:[[0.0000, 0.9988, 0.9992, 1.0000]]
8:[[0.9986, 0.0000, 0.0000, 0.0004]]
9:[[0.9992, 0.0010, 0.0000, 0.9996]]
```

_412_3.py 예제 출력 결과와 같은 것을 비교합니다.

```
tensor([[    0.0012,     0.0007,     0.0011,     0.0008],
        [    0.0006,     0.0010,     0.0007,     0.9993],
        [    0.0008,     0.0000,     0.9994,     0.0002],
        [    0.0000,     0.0009,     0.9996,     0.9997],
        [    0.0010,     0.9990,     0.0007,     0.0006],
        [    0.0004,     0.9990,     0.0002,     0.9992],
        [    0.0000,     0.9998,     0.9991,     0.0000],
        [    0.0000,     0.9988,     0.9992,     1.0000],
        [    0.9986,     0.0000,     0.0000,     0.0004],
        [    0.9992,     0.0010,     0.0000,     0.9996]])
```

아두이노 DNN 살펴보기

여기서는 아두이노 스케치로 변환된 모델을 이용하여 아두이노에서 예측을 수행해 봅니다. 다음은 아두이노 스케치에 미리 구성된 인공 신경망으로 앞에서 학습된 모델의 구조와 같은 형태입니다. 신경망은 Eigen 행렬을 이용하여 구현하였습니다. Eigen 행렬에 대한 내용은 부록을 참고합니다. 다음 소스에 대한 자세한 설명은 따로 하지 않습니다.

```
01 #include <iostream>
02 #include <ArduinoEigen.h>
03 #include "mydnn.h"
04 #include "model_data.h"
05
06 using namespace std;
07 using namespace Eigen;
08
09 IOFormat HeavyFmt(StreamPrecision, 0, ", ", ",\n", "[", "]", "[", "]");
10
11 const int NUM_PATTERN = 10;
12 const int NUM_X = 7;
13 const int NUM_H = 16;
14 const int NUM_M = 16;
15 const int NUM_Y = 4;
16
17 double xs[NUM_PATTERN][NUM_X] = {
18       { 1, 1, 1, 1, 1, 1, 0 },  // 0
19       { 0, 1, 1, 0, 0, 0, 0 },  // 1
20       { 1, 1, 0, 1, 1, 0, 1 },  // 2
21       { 1, 1, 1, 1, 0, 0, 1 },  // 3
22       { 0, 1, 1, 0, 0, 1, 1 },  // 4
23       { 1, 0, 1, 1, 0, 1, 1 },  // 5
24       { 0, 0, 1, 1, 1, 1, 1 },  // 6
25       { 1, 1, 1, 0, 0, 0, 0 },  // 7
26       { 1, 1, 1, 1, 1, 1, 1 },  // 8
27       { 1, 1, 1, 0, 0, 1, 1 }   // 9
28 };
29 double ys[NUM_PATTERN][NUM_Y];
30
31 MatrixXd X, H, M, Y;
32 MatrixXd WH(NUM_X, NUM_H), BH(1, NUM_H);
33 MatrixXd WM(NUM_H, NUM_M), BM(1, NUM_M);
34 MatrixXd WY(NUM_M, NUM_Y), BY(1, NUM_Y);
```

```
35
36 void dnn_test() {
37
38       WH = Map<Matrix<double,NUM_X,NUM_H,RowMajor>>(_W1);
39       BH = Map<MatrixXd>(_B1, 1, NUM_H);
40       WM = Map<Matrix<double,NUM_H,NUM_M,RowMajor>>(_W2);
41       BM = Map<MatrixXd>(_B2, 1, NUM_M);
42       WY = Map<Matrix<double,NUM_M,NUM_Y,RowMajor>>(_W3);
43       BY = Map<MatrixXd>(_B3, 1, NUM_Y);
44
45       long start = micros();
46       for(int pc=0;pc<NUM_PATTERN;pc++) {
47
48                X = Map<MatrixXd>(xs[pc], 1, NUM_X);
49
50                relu_f(H = X*WH + BH);
51                relu_f(M = H*WM + BM);
52                sigmoid_f(Y = M*WY + BY);
53
54                Map<MatrixXd>(ys[pc], 1, NUM_Y) = Y;
55
56       }
57       long end = micros();
58       Serial.println(end-start);
59
60       cout << fixed;
61       cout.precision(4);
62
63       for(int pc=0;pc<NUM_PATTERN;pc++) {
64                Y = Map<MatrixXd>(ys[pc], 1, NUM_Y);
65                cout << pc << ":" << Y.format(HeavyFmt) << endl;
66       }
67
68 }
69
70 void setup() {
71
72       Serial.begin(115200);
73       delay(1000);
74
```

```
75        dnn_test();
76
77 }
78
79 void loop() {}
```

38, 40, 42 : Map 클래스를 이용하여 model_data.cpp 파일에 있는 _W1, _W2, _W3 배열의 값을 WH, WM, WY 행렬로 복사합니다.

39, 41, 43 : Map 클래스를 이용하여 model_data.cpp 파일에 있는 _B1, _B2, _B3 배열의 값을 BH, BM, BY 행렬로 복사합니다.

48 : Map 클래스를 이용하여 xs[pc] 배열의 값을 X 행렬로 복사합니다.

50~52 : 순전파를 수행합니다. 예측값은 Y에 저장됩니다.

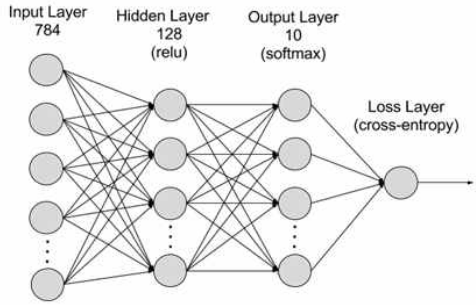
54 : Map 클래스를 이용하여 Y 행렬의 노드값을 ys[pc] 배열로 복사합니다.

03 딥러닝 활용 예제 살펴보기

여기서는 패션 MNIST 데이터 셋을 사용해서 인공 신경망을 학습시켜봅니다. 패션 MNIST 데이터 셋은 10개의 범주(category)와 70,000개의 흑백 이미지로 구성됩니다. 패션 MNIST 데이터 셋의 그림은 28x28 픽셀의 해상도를 가지며 다음 그림처럼 신발, 옷, 가방 등의 품목을 나타냅니다.



다음과 같은 모양의 인공 신경망을 구성하고 학습시켜 봅니다.



관련 라이브러리 설치하기

먼저 팅커 보드에 관련 라이브러리를 설치합니다. 다음 명령을 차례대로 실행합니다.

```
$ sudo pip3 install torchvision
$ sudo pip3 install pillow==6.2.1
$ time sudo pip3 install matplotlib==3.4
$ sudo pip3 install tqdm
```

*** matplotlib 라이브러리는 설치 시간이 6분 정도 걸립니다.

FashionMNIST 데이터 읽어오기

여기서는 FashionMNIST 데이터를 읽어와 화면에 데이터 일부분을 출력해 봅니다.

1. 다음과 같이 예제를 작성합니다.

_633.py

```
01 import torch
02 from torchvision import datasets, transforms
03 import matplotlib.pyplot as plt
04
05 train_data = datasets.FashionMNIST(
06         root='data',
07         train=True,
08         download=True,
09         transform=transforms.ToTensor())
10 test_data = datasets.FashionMNIST(
11         root='data',
12         train=False,
13         download=True,
14         transform=transforms.ToTensor())
15
16 labels_map = {
17         0:'t-shirt/top',
18         1:'trouser',
19         2:'pullover',
20         3:'dress',
21         4:'coat',
22         5:'sandal',
23         6:'shirt',
24         7:'sneaker',
25         8:'bag',
26         9:'ankle boot'}
27
28 figure = plt.figure(figsize=(10,10))
29 ROWS,COLS = 5,6
30 for i in range(1,COLS*ROWS+1):
31         rand_i = torch.randint(len(train_data),size=(1,)).item()
32         image,label = train_data[rand_i]
33         figure.add_subplot(ROWS,COLS,i)
```

```
34          plt.title(labels_map[label])
35          plt.axis('off')
36          plt.imshow(torch.permute(image,(1,2,0)),cmap='gray')
37  plt.show()
```

2 : torchvision 라이브러리에서 datasets, transforms 하위 모듈을 불러옵니다. datasets 모듈은 이미지 데이터세트 모음으로 MNIST, FashionMNIST, CIFAR, COCO, Flicker, ImageNet, UCF101등의 데이터를 가지고 있습니다. transforms 모듈은 이미지 데이터 세트에 쓸 수 있는 여러 가지 변환 필터를 가지고 있습니다. 예를 들어, 텐서 변환, 크기 조절, 밝기 조절 등의 기능이 있습니다.

3 : import문을 이용하여 matplotlib.pyplot 모듈을 plt라는 이름으로 불러옵니다. 여기서는 matplotlib.pyplot 모듈을 이용하여 28~31줄에서 FashionMNIST 데이터 셋 그림을 그립니다.

5~9 : datasets.FashionMNIST 객체를 생성하여 훈련용 데이터세트를 불러옵니다. root 매개변수는 데이터가 저장된 디렉터리를 지정합니다. train은 True일 경우 훈련용 데이터세트를 불러오고, False일 경우 시험용 데이터세트를 불러옵니다. download는 True일 경우 인터넷에서 데이터세트를 다운로드 받습니다. 데이터세트가 이미 다운로드되어 있을 경우 다시 다운로드하지는 않습니다. transform은 데이터세트의 기본 이미지인 PIL 이미지를 변환할 함수를 지정합니다. 여기서는 텐서로 변환하는 함수를 인자로 줍니다.

10~14 : datasets.FashionMNIST 객체를 생성하여 시험용 데이터를 불러옵니다.

16~26 : 패션 MNIST 데이터 셋을 이루는 품목의 종류는 10가지로 구성되며, 품목의 라벨은 숫자 0~9로 구성됩니다. 여기서는 품목의 해당 라벨 값에 품목의 이름을 대응시킵니다. labels_msp 변수를 선언한 후, dict를 이용하여 품목의 이름 10가지로 초기화합니다. 예를 들어, 품목의 라벨 값 0은 'T-shirt/top'을 의미하며, 9는 'Ankle boot'를 의미합니다. 34줄에서 사용합니다.

28~37 : 훈련용 데이터 30개를 임의로 뽑아 화면에 뿌려줍니다.

28 : plt.figure 함수를 호출하여 Figure 객체를 생성합니다.

29 : ROWS, COLS 변수를 선언하여 5, 6으로 초기화합니다.

31 : train_data 개수(6만개) 범위 내에서 임의의 정수를 만들어 rand_i 변수에 할당합니다. 0~59999 사이의 숫자 중 하나를 임의로 뽑아냅니다.

32 : 임의로 뽑아낸 rand_i에 해당하는 훈련용 그림과 라벨을 가져옵니다.

33 : figure.add_subplot 함수를 호출하여 subplot을 만듭니다. ROWS행 COLS열 중 i 번째에 subplot을 생성합니다.

34 : lables_map dict를 이용하여 subplot에 제목을 설정합니다.

35 : subplot 그림을 그릴 때 축을 표시하지 않습니다.

36 : plt.imshow 함수를 호출하여 image 그림을 subplot에 그립니다. torch.permute 함수를 호출하여 image 데이터를 화면에 출력할 수 있도록 모양을 변경합니다. 그림은 흑백으로 표시되도록 합니다.

37 : plt.show 함수를 호출하여 subplot에 그린 그림을 화면에 표시합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _633.py
```

다음은 실행 결과 화면입니다.



다음과 같이 data 디렉터리에 다운로드된 데이터세트를 확인합니다.

```
linaro@linaro-alip:~/pyLabs$ ls data/FashionMNIST/raw/ -l
total 53672
-rw-r--r-- 1 linaro linaro  7840016 Feb 22 12:48 t10k-images-idx3-ubyte
-rw-r--r-- 1 linaro linaro    10008 Feb 22 12:48 t10k-labels-idx1-ubyte
-rw-r--r-- 1 linaro linaro 47040016 Feb 22 12:48 train-images-idx3-ubyte
-rw-r--r-- 1 linaro linaro    60008 Feb 22 12:48 train-labels-idx1-ubyte
```

DataLoader 적용하기

여기서는 읽어온 데이터세트를 DataLoader 객체에 넘겨줍니다. DataLoader는 torch.utils.data 모듈에 있는 객체로 데이터세트를 섞어서 작은 묶음으로 나누어주는 역할을 합니다. 이 작은 묶음을 미니배치라고 합니다. 이 미니배치를 인공 신경망 학습 시 차례대로 넘겨줍니다.

1. 다음과 같이 예제를 수정합니다.

_633_2.py

```
01 import torch
02 from torch.utils.data import DataLoader
03 from torchvision import datasets, transforms
04
05 train_data = datasets.FashionMNIST(
06       root='data',
07       train=True,
08       download=True,
09       transform=transforms.ToTensor())
10 test_data = datasets.FashionMNIST(
11       root='data',
12       train=False,
13       download=True,
14       transform=transforms.ToTensor())
15
16 train_loader = DataLoader(
17       train_data,
18       batch_size=32,
19       shuffle=True)
20 test_loader = DataLoader(
21       test_data,
22       batch_size=32,
23       shuffle=True)
24
25 image,label = next(iter(train_loader))
26 print(image.shape, label.shape)
```

2 : torch.utils.data 모듈로부터 DataLoader를 불러옵니다. DataLoader는 입력받은 데이터세트를 for문에서 미니배치로 잘라서 내어주고, 매 회기마다 데이터를 섞어주는 역할을 합니다.

16~19 : DataLoader 객체를 생성합니다. 여기서 DataLoader 객체는 훈련용 데이터를 받아 32개의 미니배치로 자르고 매 회기마다 데이터를 섞어줍니다. 데이터를 섞어주면 인공 신경망 훈련 시 과적합을 막아줍니다.
20~23 : 시험용 데이터를 받는 DataLoader 객체를 생성합니다.
25 : 미니배치 샘플 하나를 뽑아봅니다. iter 함수를 호출하여 train_loader의 iterator 객체를 얻어온 후, next 함수를 호출하여 iterator 객체가 내어주는 항목을 하나 얻어옵니다.
26 : image와 label의 모양을 출력합니다.

2. 다음과 같이 예제를 실행합니다.

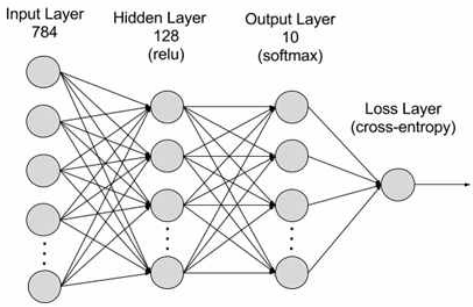
```
$ python _633_2.py
```

다음은 실행 결과 화면입니다.

```
torch.Size([32, 1, 28, 28]) torch.Size([32])
```

인공 신경망 구성하기

여기서는 다음과 같이 인공 신경망을 구성합니다.



1. 다음과 같이 예제를 수정합니다.

_633_3.py

```
01 import torch
02 import torch.nn as nn
03 from torch.utils.data import DataLoader
04 from torchvision import datasets, transforms
05
06 train_data = datasets.FashionMNIST(
07       root='data',
08       train=True,
09       download=True,
10       transform=transforms.ToTensor())
11 test_data = datasets.FashionMNIST(
12       root='data',
13       train=False,
14       download=True,
15       transform=transforms.ToTensor())
16
17 train_loader = DataLoader(
18       train_data,
19       batch_size=32,
20       shuffle=True)
21 test_loader = DataLoader(
22       test_data,
23       batch_size=32,
24       shuffle=True)
25
```

```
26 model = nn.Sequential(
27       nn.Flatten(),
28       nn.Linear(28*28,128),
29       nn.ReLU(),
30       nn.Linear(128,10)
31 )
32 print(model)
```

2 : torch.nn 라이브러리를 nn으로 불러옵니다.

26~30 : 인공 신경망을 구성합니다.

27 : nn.Flatten() 객체을 인공 신경망의 앞단에 추가하여 입력된 데이터를 평평하게 만듭니다. nn.Sequential 객체와 같이 사용합니다.

32 : model을 출력합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _633_3.py
```

다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=128, bias=True)
  (2): ReLU()
  (3): Linear(in_features=128, out_features=10, bias=True)
)
```

인공 신경망 학습하기

여기서는 인공 신경망 학습 함수를 정의한 후, 인공 신경망을 학습시켜 봅니다.

1. 다음과 같이 예제를 수정합니다.

_633_4.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04 from torch.utils.data import DataLoader
05 from torchvision import datasets, transforms
06 from tqdm import tqdm
07
08 train_data = datasets.FashionMNIST(
09     root='data',
10     train=True,
11     download=True,
12     transform=transforms.ToTensor())
13 test_data = datasets.FashionMNIST(
14     root='data',
15     train=False,
16     download=True,
17     transform=transforms.ToTensor())
18
19 train_loader = DataLoader(
20     train_data,
21     batch_size=32,
22     shuffle=True)
23 test_loader = DataLoader(
24     test_data,
25     batch_size=32,
26     shuffle=True)
27
28 model = nn.Sequential(
29     nn.Flatten(),
30     nn.Linear(28*28,128),
31     nn.ReLU(),
32     nn.Linear(128,10)
33 )
```

```
34
35 loss_fn = nn.CrossEntropyLoss()
36 optimizer = optim.Adam(model.parameters(), lr=0.001)
37
38 def model_fit(model, loss_fn, optimizer, data_loader):
39     sum_loss,hit = 0,0
40     progress_bar = tqdm(data_loader)
41     for image, label in progress_bar:
42         pred = model(image) # 1공식, 순전파
43         loss = loss_fn(pred, label) # 2공식, 오차계산
44         optimizer.zero_grad()
45         loss.backward() # 6공식, 오차역전파
46         optimizer.step() # 7공식, 학습
47
48         _,pred = pred.max(dim=1)
49         hit += pred.eq(label).sum().item()
50         sum_loss += loss.item()*image.size(0)
51
52     avg_loss = sum_loss/len(data_loader.dataset)
53     avg_acc = hit/len(data_loader.dataset)
54     return avg_loss, avg_acc
55
56 for epoch in range(5):
57     train_loss, train_acc = model_fit(
58         model, loss_fn, optimizer, train_loader)
59     print(f'epoch {epoch+1:02d}, loss: {train_loss:.5f}, acc: {train_acc:.5f}')
```

3 : torch.optim 모듈을 optim으로 불러옵니다.

6 : tqdm 라이브러리로부터 tqdm 모듈을 불러옵니다.

35 : 오차 함수를 설정합니다. CrossEntropyLoss 객체를 사용합니다.

36 : 최적화 함수를 설정합니다. Adam 객체를 사용합니다. 학습률은 0.001로 설정합니다.

38~54 : model_fit 함수를 정의합니다. model_fit 함수는 인공 신경망 학습 함수입니다.

39 : sum_loss, hit 변수를 선언한 후, 0, 0으로 초기화합니다.

40 : tqdm 객체를 생성하며, data_loader를 입력으로 줍니다. tqdm 객체는 학습 진행 상태를 시각적으로 보여줍니다.

41~50 : progress_bar로부터 image, label 데이터를 가져와 인공 신경망 학습을 진행합니다. image, label은 21줄에서 설정된 32개의 데이터로 구성됩니다. 즉, 32개의 이미지와 32개의 라벨이 학습에 사용됩니다.

48 : 예측 값의 최대 항목 위치 값을 얻어와 pred 변수에 할당합니다.

49 : 32개의 데이터에 예측값과 라벨값이 같은 데이터의 개수를 얻어와 hit 변수에 누적해 줍니다.

50 : loss.item()은 32개의 입력 데이터의 평균 오차를 내어주기 때문에 이미지 개수인

32만큼 곱해서 sum_loss에 더해줍니다.

52 : sum_loss는 6만개 데이터 전체에 대한 오차 합이고 len(train_loader.dataset)은 6만이 되어 avg_loss는 6만개 데이터에 대한 평균 오차가 됩니다.

53 : 전체 데이터 6만개 중 예측값과 목표값이 같은 데이터의 개수의 비율을 구해 avg_acc 변수에 할당합니다.

54 : 평균 오차값과 평균 정답률을 내어줍니다.

56~59 : 학습을 5회 수행합니다.

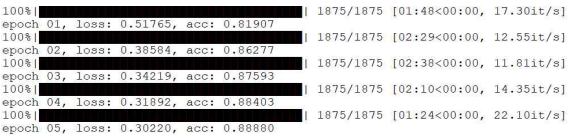
57~58 : model_fit 함수를 호출하여 인공 신경망을 학습시킵니다.

59 : 학습 결과를 출력합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _633_4.py
```

다음은 실행 결과 화면입니다.



학습된 인공 신경망 평가하기

여기서는 인공 신경망 평가 함수를 정의한 후, 학습된 인공 신경망을 평가해 봅니다.

1. 다음과 같이 예제를 수정합니다.

_633_5.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04 from torch.utils.data import DataLoader
05 from torchvision import datasets, transforms
06 from tqdm import tqdm
07
08 train_data = datasets.FashionMNIST(
09         root='data',
10         train=True,
11         download=True,
12         transform=transforms.ToTensor())
13 test_data = datasets.FashionMNIST(
14         root='data',
15         train=False,
16         download=True,
17         transform=transforms.ToTensor())
18
19 train_loader = DataLoader(
20         train_data,
21         batch_size=32,
22         shuffle=True)
23 test_loader = DataLoader(
24         test_data,
25         batch_size=32,
26         shuffle=True)
27
28 model = nn.Sequential(
29         nn.Flatten(),
30         nn.Linear(28*28,128),
31         nn.ReLU(),
32         nn.Linear(128,10)
33 )
```

```
34
35 loss_fn = nn.CrossEntropyLoss()
36 optimizer = optim.Adam(model.parameters(), lr=0.001)
37
38 def model_fit(model, loss_fn, optimizer, data_loader):
39       sum_loss,hit = 0,0
40       progress_bar = tqdm(data_loader)
41       for image, label in progress_bar:
42                pred = model(image) # 1공식, 순전파
43                loss = loss_fn(pred, label) # 2공식, 오차계산
44                optimizer.zero_grad()
45                loss.backward() # 6공식, 오차역전파
46                optimizer.step() # 7공식, 학습
47
48                _,pred = pred.max(dim=1)
49                hit += pred.eq(label).sum().item()
50                sum_loss += loss.item()*image.size(0)
51
52       avg_loss = sum_loss/len(data_loader.dataset)
53       avg_acc = hit/len(data_loader.dataset)
54       return avg_loss, avg_acc
55
56 def model_evaluate(model, loss_fn, data_loader):
57       with torch.no_grad():
58                sum_loss,hit = 0,0
59                for image, label in data_loader:
60                         pred = model(image)
61                         loss = loss_fn(pred, label)
62
63                         _,pred = pred.max(dim=1)
64                         hit += pred.eq(label).sum().item()
65                         sum_loss += loss.item()*image.size(0)
66
67       avg_loss = sum_loss/len(data_loader.dataset)
68       avg_acc = hit/len(data_loader.dataset)
69       return avg_loss, avg_acc
70
71 for epoch in range(5):
72       train_loss,train_acc = model_fit(
73                model, loss_fn, optimizer, train_loader)
```

```
74        print(f'epoch {epoch+1:02d}, loss: {train_loss:.5f}, acc: {train_acc:.5f}')
75
76        valid_loss,valid_acc = model_evaluate(
77                  model, loss_fn, test_loader)
78        print('  '*8, f'vloss: {valid_loss:.5f},vacc: {valid_acc:.5f}')
79
80  torch.save(model, 'fashion_model.pth')
```

56~69 : model_evaluate 함수를 정의합니다. model_evaluate 함수는 인공 신경망 평가 함수입니다.

57 : torch.no_grad 객체를 생성하여 58~65줄 수행 시 역전파 계산을 비활성화합니다. 인공 신경망 학습이 끝나면 역전파 계산을 수행하지 않습니다.

58 : sum_loss, hit 변수를 선언한 후, 0, 0으로 초기화합니다.

76~77 : model_evaluate 함수를 호출하여 학습된 인공 신경망을 평가합니다.

78 : 평가 결과를 출력합니다.

80 : 학습된 모델을 저장합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _633_5.py
```

다음은 실행 결과 화면입니다.

```
100%|██████████| 1875/1875 [02:03<00:00, 15.24it/s]
epoch 01, loss: 0.51496, acc: 0.81942
        vloss: 0.42516,vacc: 0.84900
100%|██████████| 1875/1875 [01:22<00:00, 22.61it/s]
epoch 02, loss: 0.37993, acc: 0.86163
        vloss: 0.38785,vacc: 0.86080
100%|██████████| 1875/1875 [01:22<00:00, 22.78it/s]
epoch 03, loss: 0.34003, acc: 0.87613
        vloss: 0.38081,vacc: 0.86810
100%|██████████| 1875/1875 [02:19<00:00, 13.46it/s]
epoch 04, loss: 0.31849, acc: 0.88393
        vloss: 0.36332,vacc: 0.86870
100%|██████████| 1875/1875 [02:43<00:00, 11.49it/s]
epoch 05, loss: 0.30029, acc: 0.89023
        vloss: 0.35294,vacc: 0.87180
```

3. 학습이 끝난 후, 다음과 같이 fashion_model.pth 파일이 생성된 것을 확인합니다.

```
linaro@linaro-alip:~/pyLabs$ ls -l fashion_model.pth
-rw-r--r-- 1 linaro linaro 409731 Feb 22 14:51 fashion model.pth
```

학습된 인공 신경망 불러와 예측하기

여기서는 학습된 인공 신경망을 불러와 예측을 수행해 봅니다.

1. 다음과 같이 예제를 수정합니다.

_633_6.py

```
01 import torch
02 from torch.utils.data import DataLoader
03 from torchvision import datasets, transforms
04 import matplotlib.pyplot as plt
05 import numpy as np
06
07 test_data = datasets.FashionMNIST(
08     root='data',
09     train=False,
10     download=True,
11     transform=transforms.ToTensor())
12
13 test_loader = DataLoader(
14     test_data,
15     batch_size=32,
16     shuffle=True)
17
18 model = torch.load('fashion_model.pth')
19
20 label_tags = {
21     0: 'T-Shirt',
22     1: 'Trouser',
23     2: 'Pullover',
24     3: 'Dress',
25     4: 'Coat',
26     5: 'Sandal',
27     6: 'Shirt',
28     7: 'Sneaker',
29     8: 'Bag',
30     9: 'Ankle Boot'
31 }
32
33 fig = plt.figure(figsize=(10,10))
```

```
34 ROWS,COLS=6,6
35 for i in range(1, COLS*ROWS+1):
36     data_idx = np.random.randint(len(test_loader))
37     input_img = test_data[data_idx][0].unsqueeze(dim=0)
38 
39     output = model(input_img)
40     _, argmax = torch.max(output, 1)
41     pred = label_tags[argmax.item()]
42     label = label_tags[test_data[data_idx][1]]
43 
44     fig.add_subplot(ROWS, COLS, i)
45     if pred == label:
46         plt.title(pred)
47         cmap = 'Greys'
48     else:
49         plt.title(pred + '(' +  label + ')')
50         cmap = 'Oranges'
51     plot_img = test_data[data_idx][0][0,:,:]
52     plt.imshow(plot_img, cmap=cmap)
53     plt.axis('off')
54 
55 plt.show()
```

1~5 : 관련 라이브러리를 불러옵니다.
7~11 : FashionMNIST 시험 데이터세트를 불러옵니다.
13~16 : 시험 데이터세트를 DataLoader에 넘겨줍니다.
18 : torch.load 함수를 호출하여 저장된 인공 신경망 모델을 불러옵니다.
20~31 : Fashion MNIST의 라벨값과 종류를 연결하는 dict를 생성합니다.
33~55 : 시험용 데이터 36개를 임의로 뽑아 예측을 수행하고 예측 결과와 실제 값을 비교하여 화면에 뿌려줍니다.
37 : 예측을 수행하기 위해 unsqueeze 함수를 호출하여 데이터의 모양을 바꿔줍니다.
39 : 예측을 수행합니다.
40 : 예측 결과 중 최대값을 갖는 항목의 위치값을 argmax로 얻어옵니다.
41 : 예측값에 대한 항목 이름을 가져옵니다.
42 : 실제값에 대한 항목 이름을 가져옵니다.
44~52 : 맞으면 희색으로 틀리면 주황색으로 그림을 표시합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _633_6.py
```

다음은 실행 결과 화면입니다.



회색으로 표시된 그림들은 맞게 예측한 것이고, 주황색으로 표시한 그림들은 틀린 예측입니다.

인공 신경망 클래스 정의해 보기

일반적으로 PyTorch 인공 신경망은 클래스로 정의합니다. 여기서는 인공 신경망을 클래스로 정의하고 테스트해 봅니다.

1. _633_5.py 예제를 다음과 같이 수정합니다. 28~33줄을 다음 예제로 대체합니다.

_633_7.py

```
28 class MYDNN(nn.Module):
29     def __init__(self):
30         super(MYDNN, self).__init__()
31         self.flatten = nn.Flatten()
32         self.fc1 = nn.Linear(28*28,128)
33         self.av1 = nn.ReLU()
34         self.fc2 = nn.Linear(128,10)
35     def forward(self,x):
36         x = self.flatten(x)
37         x = self.fc1(x)
38         x = self.av1(x)
39         x = self.fc2(x)
40         return x
41 model = MYDNN()
42 print(model)
```

28~41 : MYDNN 클래스를 정의합니다. nn.Module 클래스를 상속합니다.
29~34 : __init__ 함수를 정의합니다. 사용할 신경망 층을 결정합니다.
35~40 : forward 함수를 정의합니다. 순전파 과정을 구현합니다.
41 : MYDNN 모델을 생성합니다.
42 : 모델을 출력합니다.

2. 다음과 같이 예제를 실행합니다.

```
$ python _633_7.py
```

다음은 실행 결과 화면입니다.

```
MYDNN(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (av1): ReLU()
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
```

```
100%|████████████████████████████████| 1875/1875 [02:03<00:00, 15.24it/s]
epoch 01, loss: 0.51496, acc: 0.81942
        vloss: 0.42516,vacc: 0.84900
100%|████████████████████████████████| 1875/1875 [01:22<00:00, 22.61it/s]
epoch 02, loss: 0.37993, acc: 0.86163
        vloss: 0.38785,vacc: 0.86080
100%|████████████████████████████████| 1875/1875 [01:22<00:00, 22.78it/s]
epoch 03, loss: 0.34003, acc: 0.87613
        vloss: 0.38081,vacc: 0.86810
100%|████████████████████████████████| 1875/1875 [02:19<00:00, 13.46it/s]
epoch 04, loss: 0.31849, acc: 0.88393
        vloss: 0.36332,vacc: 0.86870
100%|████████████████████████████████| 1875/1875 [02:43<00:00, 11.49it/s]
epoch 05, loss: 0.30029, acc: 0.89023
        vloss: 0.35294,vacc: 0.87180
```

CNN 신경망 구성해 보기

여기서는 간단히 CNN 신경망 구성을 살펴봅니다. 설명은 따로 하지는 않습니다.

nn.Sequential 객체를 이용하여 모델을 구성할 경우 다음과 같습니다.

```
model = nn.Sequential(
    nn.Conv2d(1,32,5,padding=2),
    nn.MaxPool2d(2),
    nn.Conv2d(32,64,5,padding=2),
    nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(64*7*7,1024),
    nn.ReLU(),
    nn.Linear(1024,10)
)
```

클래스 정의는 다음과 같습니다.

```
class MYCNN(nn.Module):
    def __init__(self):
            super(MYCNN, self).__init__()
            self.conv1 = nn.Conv2d(1,32,5,padding=2)
            self.pool1 = nn.MaxPool2d(2)
            self.conv2 = nn.Conv2d(32,64,5,padding=2)
            self.pool2 = nn.MaxPool2d(2)
            self.flatten = nn.Flatten()
            self.fc1 = nn.Linear(64*7*7,1024)
            self.av1 = nn.ReLU()
            self.fc2 = nn.Linear(1024,10)
    def forward(self,x):
            x = self.conv1(x)
            x = self.pool1(x)
            x = self.conv2(x)
            x = self.pool2(x)
            x = self.flatten(x)
            x = self.fc1(x)
            x = self.av1(x)
            x = self.fc2(x)
            return x
model = MYCNN()
```

CNN 신경망의 경우 테스트를 따로 수행하지 않습니다. 학습 시간이 기본 신경망(Fully Connected Neural Network)에 비해 많이 걸립니다. PC에 파이토치 환경을 구성하고 테스트하면 시간을 단축시킬 수 있습니다.